



Tomato



Banana

MULTI-FRUIT

QUALITY CLASSIFICATION

Using Convolutional Neural Networks

PyTorch

4 Classes

CNN

Kaggle Dataset

Deep Learning · Computer Vision · Agriculture AI

Introduction

OVERVIEW

The Problem

1. Manual inspection of fruit quality is slow and subjective
2. Human graders suffer fatigue leading to inconsistent grades
3. No scalable automated system exists for multi-fruit grading
4. Post-harvest losses are high due to poor quality sorting

Our Solution

- ✓ CNN classifies 4 fruit quality classes in real-time
- ✓ Covers 2 fruits: Tomatoes & Bananas (Ripe / Bad)
- ✓ Custom dataset built and uploaded to Kaggle
- ✓ Single image → class label + confidence, saved as .pth

"One image → four quality classes across two fruits, instantly."

Applications & Motivation

WHY IT MATTERS



Food Processing

Automated multi-fruit sorting on conveyor belts



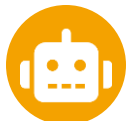
Retail Quality

Shelf-life prediction and stock management



Farm & Export

Pre-shipment grading to meet quality standards



Agri-Robotics

Vision module for fruit-picking robots



Mobile App

Farmer-facing field quality check tool



Waste Reduction

~1.3B tonnes of food wasted globally per year

Dataset Collection Details

CUSTOM DATASET

Source

Custom dataset — uploaded to Kaggle
muhammadanasfaisal / pai-project-dataseet-2

Split Strategy

Split	Train	Test
PyTorch v3	80%	20%

Config

Image Size: 128 × 128 px | Format: RGB
Stratified split ensures class balance

4 Quality Classes



Ripe Tomato

Label 0



Bad Tomato

Label 1



Ripe Banana

Label 2



Bad Banana

Label 3

Pre-Processing Pipeline

DATA PREP

01

Custom Dataset Class

CustomDataset(Dataset) wraps image paths + labels; loads on demand via PIL

03

Normalization

ToTensor() scales pixel values $[0, 255] \rightarrow [0.0, 1.0]$ automatically

05

Label Mapping

ripe_tomato=0 bad_tomato=1 ripe_banana=2
bad_banana=3

02

Resize

All images resized to 128×128 px for uniform CNN input shape

04

Data Augmentation

Flip · Rotation $\pm 20^\circ$ · ColorJitter (train only) — test set: resize only

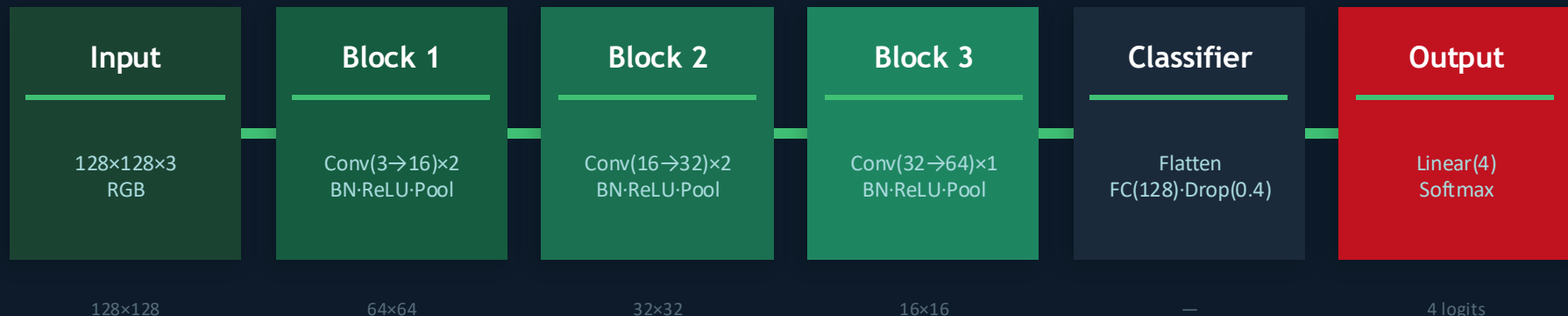
06

DataLoader

Batch size 30, shuffle=True for train; stratified 80/20 split via sklearn

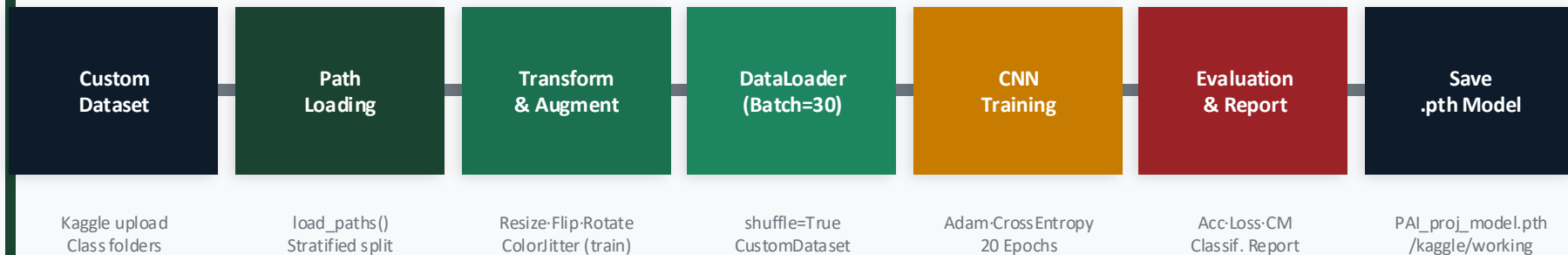
Model Architecture

TinyVGG-style CNN · PyTorch · 4-Class Output



Complete System Pipeline

METHODOLOGY



⚙️ Key Configuration

Framework:	PyTorch	IMG Size:	128×128	Batch Size:	30
Epochs:	20	LR:	0.001	Optimizer:	Adam
Loss:	CrossEntropyLoss	Classes:	4	Split:	80 / 20

Experimental Details

HYPERPARAMETERS

Parameter	Value / Setting	Notes
Framework	PyTorch	GPU auto-detected (CUDA/CPU)
Dataset	Custom Kaggle Dataset	muhammadanasfaisal/pai-project-dataseet-2
Image Size	128 × 128 px	RGB colour, resized via transforms
Batch Size	30	DataLoader shuffle=True for train
Epochs	20	Loss tracked per epoch
Learning Rate	0.001	Adam adaptive optimiser
Loss Function	CrossEntropyLoss	Multi-class, PyTorch built-in
Dropout Rate	0.4	Applied in classifier head
Train / Test	80% / 20%	Stratified split via sklearn
Augmentation	Flip, Rotate±20°, ColorJitter	Train only; test: Resize + ToTensor
Model Output	4 classes	ripe_tomato · bad_tomato · ripe_banana · bad_banana
Saved Model	PAI_proj_model.pth	/kaggle/working — state_dict format

Results

QUANTITATIVE

x%

Test Accuracy

on held-out 20% set

X

Final Loss

training CrossEntropy

4

Classes

Tomato & Banana

20

Epochs

PyTorch training loop

CLASSIFICATION REPORT

Class	Precision	Recall	F1-Score	Support
🍅 Ripe Tomato (0)	[x.xx]	[x.xx]	[x.xx]	[N]
🍅 Bad Tomato (1)	[x.xx]	[x.xx]	[x.xx]	[N]
🍌 Ripe Banana (2)	[x.xx]	[x.xx]	[x.xx]	[N]
🍌 Bad Banana (3)	[x.xx]	[x.xx]	[x.xx]	[N]
Weighted Avg	[x.xx]	[x.xx]	[x.xx]	[N]

4x4 Confusion Matrix



Conclusion

[Achievements](#) · [Limitations](#) · [Future Work](#)

✅ Achievements

- ▶ End-to-end 4-class fruit quality CNN pipeline
- ▶ Covers two fruits: Tomatoes + Bananas
- ▶ Custom dataset created and hosted on Kaggle
- ▶ Real-time inference saved as .pth

⚠️ Limitations

- ▶ Sensitive to lighting / background variation
- ▶ Only 2 fruit types — needs broader coverage
- ▶ No validation set — can't track overfitting per epoch
- ▶ No live camera stream integrated

🚀 Future Work

- ▶ Add validation split + early stopping
- ▶ Transfer learning: ResNet / EfficientNet
- ▶ Extend to more fruit types & disease categories
- ▶ Deploy as mobile app for field farmers

Scalable · Automated · Multi-Fruit Quality Intelligence



LIVE

🍅 ripe_tomato / bad_tomato

🍌 ripe_banana / bad_banana

4 Classes · Confidence %

Recorded Demo

What the demo covers:

- 1 Code structure walkthrough (dataset · model · training loop)
- 2 Loading saved model from PAI_proj_model.pth
- 3 Single-image inference on test tomato & banana images
- 4 Displaying predicted class + confidence score per image
- 5 Training loss curve & 4×4 confusion matrix review

"Image in → model predicts class + confidence %"